

Synthetic domain spaces

We define a finite discrete POMDP with scalable state and observation spaces. The goal is to create a controlled domain for evaluating the gap between the full robust value of a fixed tree policy and the projected robust value computed on sampled supports.

State space

The hidden state space is:

$$S=\{0,1,\dots,N-1\}.$$

The state index represents the underlying hidden risk level.

- Low-index states correspond to safe conditions.
- Middle-index states correspond to risky conditions.
- The highest-index state corresponds to catastrophic conditions.

For implementation, define the state partition using parameters:

- n_{safe}
- n_{risky}
- $n_{catastrophic}$
- $n_{safe} + n_{risky} + n_{catastrophic} = 1$
-

for example:

- `safe_fraction = 0.70`
- `risky_fraction = 0.29`
- `catastrophic_fraction = 0.01`
- `n_safe = int(safe_fraction * N)`
- `n_cat = max(1, int(catastrophic_fraction * N))` (keep max(1,...) so that small debugging domains still contain at least one catastrophic state.)
- `n_risky = N - n_safe - n_cat`
- `safe_states = range(0, n_safe)`
- `risky_states = range(n_safe, n_safe + n_risky)`
- `catastrophic_states = range(n_safe + n_risky, N)`

This keeps the domain interpretable while allowing the number of states N to scale.

Action space

We keep the action space small and interpretable:

$$A=\{\text{inspect},\text{proceed},\text{mitigate}, \text{abort}\}.$$

The action space is intentionally small because the main computational burden in this experiment comes from the state space, observation space, horizon, and sampled support size, not from the number of actions.

Interpretation:

- inspect: gather information through observations, with a small cost.
- proceed: take the risky/high-reward action.
- mitigate take the safe fallback action.
- abort: stop the episode now," receive safe payoff, and no future uncertainty matters

This is enough to create meaningful fixed policies while keeping the experiment focused on full-vs-projected robust evaluation.

Observation space

The observation space is:

$Z=\{0,1,\dots,M-1\}$.

The observation index represents the severity of the observed warning signal.

- Low-index observations mean the sensor indicates safe conditions.
- High-index observations mean the sensor indicates dangerous conditions

This mirrors the state-space ordering, but with a different meaning:

- state index=true hidden risk level,
- observation index=observed warning level.

The observation model $O(z|s)$ will correlate observations with hidden risk: safe states tend to produce low-warning observations, risky states tend to produce middle/high-warning observations, and catastrophic states tend to produce high-warning observations.

Important: observations are not omitted because they are intrinsically rare. In the projected tree, observations are omitted because the sampled online tree has not encountered them yet. Some observations may be unlikely under the current belief because they are associated with low-belief risky/catastrophic states, but omission is a property of the finite sampled support, not of the observation space itself.

models:

belief:

Each class receives a fixed total probability mass, for example:

- `mass_safe = 0.85`
- `mass_risky = 0.14`
- `mass_cat = 0.01`

Inside each class, the belief decreases gradually with the state index. This means lower-risk states inside the class are more likely than higher-risk states.

`b0[safe_0] > b0[safe_1] > b0[safe_2] > ...`

and similarly for risky and catastrophic states.

Implementation idea:

```
def decaying_region_belief(states, total_mass, decay_rate):
    raw_weights = np.exp(-decay_rate * np.arange(len(states)))
    probs = total_mass * raw_weights / raw_weights.sum()
    return dict(zip(states, probs))
```

Then construct the full belief:

```
b0 = np.zeros(N)
```

```
for s, p in decaying_region_belief(safe_states, mass_safe, lambda_safe).items():
    b0[s] = p
```

```
for s, p in decaying_region_belief(risky_states, mass_risky, lambda_risky).items():
    b0[s] = p
```

```
for s, p in decaying_region_belief(catastrophic_states, mass_catastrophic,
lambda_catastrophic).items():
    b0[s] = p
```

The parameters should be configurable:

- `lambda_safe = 0.03`
- `lambda_risky = 0.05`
- `lambda_catastrophic = 0.10`

Purpose:

- The total probability of being safe/risky/catastrophic is controlled.
- Within each class, lower-index states are more likely.
- The belief scales naturally as the number of states N grows.
- Catastrophic states have small total mass, but are still possible, which is important for testing omitted-support effects.

Transition model

We define the transition model over the ordered hidden state space:

- $S = \{0, 1, \dots, N-1\}$

The state index represents hidden risk level:

- low index = safer state
- high index = more dangerous state

The transition model is action-dependent:

- $T[\text{action}, s, s_{\text{next}}]$

We use four actions:

- `actions = ["inspect", "proceed", "mitigate", "abort"]`

inspect

Information-gathering action. It has a small cost, and the hidden state mostly stays the same.

Suggested transition:

- 0.90 stay at same risk level
- 0.10 move one step toward higher risk

Example:

- $T[\text{"inspect"}, s, s] += 0.90$
- $T[\text{"inspect"}, s, \min(s + 1, N - 1)] += 0.10$

proceed

Risky/high-reward action. It may push the system toward more dangerous states.

Suggested transition:

- 0.75 stay at same risk level
- 0.20 move one step toward higher risk
- 0.05 move two steps toward higher risk

Example:

- $T[\text{"proceed"}, s, s] += 0.75$
- $T[\text{"proceed"}, s, \min(s + 1, N - 1)] += 0.20$
- $T[\text{"proceed"}, s, \min(s + 2, N - 1)] += 0.05$

mitigate

Non-terminal safety action. It reduces risk but has a cost. The episode continues.

Suggested transition:

- 0.80 stay at same risk level
- 0.20 move one step toward lower risk

Example:

- $T[\text{"mitigate"}, s, s] += 0.80$
- $T[\text{"mitigate"}, s, \max(s - 1, 0)] += 0.20$

abort

Terminal fallback action. It stops the episode immediately with a safe payoff.

For abort, we do not need meaningful future transitions or observations. In implementation, either:

- `is_terminal_action("abort") = True`

or define an absorbing terminal state separately.

For the first implementation, the simpler option is:

if `action == "abort"`:

`return terminal_value`

and skip transition/observation backup after abort.

Notes

- Transitions should use boundary clipping so states remain in $[0, N-1]$.
- The probabilities above are starting defaults and should be configurable.
- The model scales automatically with N .
- The transition structure reflects the semantic risk ordering of the state space.

Observation model

We use an action-independent observation model:

- `O[s_next, z]`

Meaning:

- after the hidden state transitions to `s_next`,
- the agent receives observation `z`,
- and the observation depends only on the hidden state, not on the action.

This matches the theoretical setup where the observation model is written as:

- $P_Z(z | s_{next})$

Observation space

The observation space is:

- $Z = \{0, 1, \dots, M - 1\}$

The observation index represents warning severity:

- low `z` index = safe-looking signal
- high `z` index = dangerous-looking signal

Shape of the observation model

Each hidden state `s` has a target observation index:

- `target_z(s)`

This target increases with the risk level of the state:

- safe states -> low `target_z`
- risky states -> middle/high `target_z`
- catastrophic states -> high `target_z`

Then define the observation probabilities using a noisy distribution around the target:

- $O[s, z] \propto \exp(-\text{abs}(z - \text{target_z}(s)) / \text{obs_noise})$

After computing the raw weights, normalize over all observations:

- $O[s, :] = \text{raw_weights} / \text{raw_weights.sum}()$

Implementation idea

```
def build_observation_model(N, M, obs_noise):
```

```
    O = np.zeros((N, M))
```

```
    for s in range(N):
```

```
        # map state risk index to observation-warning index
```

```
        target = round((s / (N - 1)) * (M - 1))
```

```
        raw = np.exp(-np.abs(np.arange(M) - target) / obs_noise)
```

```
        O[s, :] = raw / raw.sum()
```

```
    return O
```

Notes

- obs_noise should be configurable.
- Smaller obs_noise means observations are more accurate.
- Larger obs_noise means observations are noisier.
- Observations are not omitted because they are intrinsically rare.
- In the projected tree, an observation is omitted only because the sampled tree has not included it yet.
- Some observations may be unlikely under the current belief, but that comes from the belief and observation model together, not from the observation space itself.

Reward model

The reward model is state-action dependent:

- $R[s, \text{action}]$

The rewards should reflect the hidden risk level of the state and the meaning of each action.

- actions = ["inspect", "proceed", "mitigate", "abort"]

inspect:

Cheap information/waiting action.

- $R[s, \text{"inspect"}] = -c_inspect$
- Suggested default: $c_inspect = 1.0$

proceed

Risky reward-seeking action.

- safe states → high positive reward

- risky states -> negative reward
- catastrophic states -> large negative reward
- Suggested defaults:
 - $r_proceed_safe = 10.0$
 - $c_proceed_risky = 20.0$
 - $c_proceed_catastrophic = 100.0$

mitigate

Non-terminal safety intervention. It reduces future risk through the transition model, but it has an immediate cost that becomes worse in riskier states.

- safe states -> small cost
- risky states -> larger cost
- catastrophic states -> large cost
- Suggested defaults:
 - $c_mitigate_safe = 2.0$
 - $c_mitigate_risky = 8.0$
 - $c_mitigate_catastrophic = 30.0$

abort

Terminal safe fallback action.

- $R[s, "abort"] = r_abort$

Suggested default:

- $r_abort = 0.0$

If the action is abort, the evaluator should stop the episode and not apply future transition/observation backups.

Purpose

This reward model creates the intended tradeoff:

- proceed is attractive when the system is safe.
- proceed is dangerous when hidden risk is high.
- mitigate is costly but safer than proceeding in risky states.
- abort gives up future reward but avoids catastrophic outcomes.
- inspect is cheap but does not directly reduce risk.

POLICY of the TREE

A1: Fixed-Policy Projected Robust Tree Evaluator

Goal

A1 is not a planner. It is a controlled evaluator used to validate the theory.

The goal is to compare:

full robust value of a fixed tree policy

vs.

projected robust value of the same fixed tree policy

This directly tests the core theory: how much error is introduced when we evaluate a robust POMDP tree only on a sampled/projected support instead of the full state-observation support.

Inputs

A1 receives:

1. A full finite-horizon tree
2. A fixed policy over the tree
3. Full state space S
4. Full observation space Z
5. Nominal transition and observation models
6. Robust uncertainty sets
7. Initial belief b_0
8. A sequence of projected supports

The policy is fixed before evaluation. That means that at every decision node, the action is already specified.

Example:

- root:
 - inspect
- after low-warning observation:
 - proceed
- after medium-warning observation:
 - mitigate
- after high-warning observation:
 - abort

Full tree

The full tree is the reference tree.

It includes:

- all states in S

- all observations in Z
- all observation branches up to horizon H
- the same fixed policy at every history

The full robust evaluator computes the root value:

- $V_{\text{full_robust_pi}}(h_0)$

This is the robust value of the fixed policy at the root.

Projected trees

Projected trees are created by restricting the full tree to smaller supports.

For each projection level m , define:

- $S_{\text{in}}[m]$ subset of S
- $Z_{\text{in}}[m]$ subset of Z

Then evaluate the same fixed policy using only this projected support.

Important:

The policy does not change across projection levels.

Only the support changes.

So if the full policy says:

- low warning -> proceed
- medium warning -> mitigate
- high warning -> abort

then the projected tree uses the same policy rule, but only on the observation branches that remain inside $Z_{\text{in}}[m]$.

Omitted states and observations are not explicitly evaluated. Their possible contribution is represented through the leakage/error certificate.

Outputs

For each projected support level, A1 should output:

1. projected robust root value
2. full robust root value
3. true projection error
4. deterministic certificate bound
5. leakage term
6. inside-support mismatch term, if available
7. validity check: $\text{true_error} \leq \text{certificate_bound}$

What A1 is meant to show

A1 should show:

1. The projected robust backup is computable.
2. The projected robust value approaches the full robust value as support grows.
3. The true error shrinks as support grows.
4. The deterministic certificate upper-bounds the true error.
5. The certificate shrinks as support grows.
6. The leakage term decreases as omitted support decreases.

Tree construction for A1

Each tree node represents a history, for example:

h_0

$h_0 \xrightarrow{\text{inspect}} z$

$h_0 \xrightarrow{\text{inspect}} z \xrightarrow{\text{mitigate}} z'$

Each node should store:

- depth
- history
- fixed action $\pi(h)$
- state support $S_{in}(h)$
- observation support $Z_{in}(h)$
- children indexed by observation

Full tree

The full tree is the reference tree.

At every node in the full tree:

- $S_{in}(h)$ = full state space S
- $Z_{in}(h)$ = full observation space Z

The full tree is built recursively up to horizon H .

At the root:

action = inspect

After a non-terminal action, create one child for every observation:

for z in Z :

child history = parent history + (action, z)

Then assign the next action using the fixed policy rule.

Fixed policy rule

The fixed policy is generated from the most recent observation warning level.
Observation indices represent warning severity:

- low z index = safe-looking signal
- high z index = dangerous-looking signal

Example rule:

```
def fixed_policy_action(depth, last_observation, M):
```

```
    if depth == 0:  
        return "inspect"
```

```
    warning = last_observation / (M - 1)
```

```
    if warning < 0.4:  
        return "proceed"  
    elif warning < 0.75:  
        return "mitigate"  
    else:  
        return "abort"
```

Terminal conditions

Stop expanding a node if:

- $depth == H$

or if:

- $action == abort$

abort is terminal, so after abort there are no future transition or observation branches.

Projected trees

Projected trees are created by restricting the full tree to smaller state and observation supports.

They are not generated independently.

For each projection level m , define:

- $S_{in}[m]$ subset of S
- $Z_{in}[m]$ subset of Z

Then construct:

```
projected_tree[m] = restrict(full_tree, S_in[m], Z_in[m])
```

The policy rule does not change across projection levels.

Only the supports change.

So if the full policy says:

- low warning -> proceed
- medium warning -> mitigate
- high warning -> abort

then the projected tree uses the same rule, but only for observation branches that remain inside $Z_{in}[m]$.

Omitted observations and states are not explicitly evaluated in the projected value. Their possible contribution is accounted for through the leakage/error certificate.

Projection levels

Use multiple projected supports to study support-size effects.

Example:

small projection:

S_{in} = first 30% of states

Z_{in} = first 40% of observations

medium projection:

S_{in} = first 60% of states

Z_{in} = first 70% of observations

large projection:

S_{in} = first 90% of states

Z_{in} = first 90% of observations

full projection:

S_{in} = all states

Z_{in} = all observations

